Trace-Based Bytecode Interpreter Visualization for Compiler Construction Education Artifact Description

Tobias Herber
Institute for System Software
Johannes Kepler University
Linz, Austria
tobias@herber.space

Markus Weninger
Institute for System Software
Johannes Kepler University
Linz, Austria
markus.weninger@jku.at

***Abstract*—Enhancing students' comprehension of bytecode generation and its interpretation is crucial, yet challenging, in compiler construction courses. Traditional approaches often emphasize theoretical concepts, making it difficult for students to grasp the inner workings of bytecode interpreters.**

To bridge this gap, we introduce a web-based tool for *trace-based bytecode interpreter visualization*, designed to enhance comprehension by providing an interactive and visually enriched learning experience. Our tool provides a side-by-side view that aligns the original high-level source code and its corresponding low-level bytecode, with arrows indicating jumps and method calls. Users can step through the executed bytecode (forwards and backwards) to see the effect of each operation. A dynamic memory visualization utilizes animations to illustrate changes in the interpreter's various memory regions and its registers. To further increase the flexibility of our tool, we developed a lightweight metalanguage that enables educators to define visualizations for arbitrary bytecode formats.

Our tool aims to bridge the gap between abstract theory and concrete execution. We demonstrate its effectiveness in various educational settings, e.g., how it can help educators improve their live teaching and how it facilitates student self-study.

***Index Terms*—Compiler Construction, Bytecode Interpreter, Visualization, Interactive Learning, Debugging, Trace-Based Debugging, Computer Science Education**

I. Introduction

This document accompanies the paper "*Trace-Based Bytecode Interpreter Visualization for Compiler Construction Education*", by Herber and Weninger, and serves as a supplement describing the artifacts developed as part of the work [1]. It provides a technical overview of the visualization tool introduced in the paper, along with concrete examples and demonstration files that showcase its functionality in practice.

While the main paper focuses on the conceptual design, motivation, and educational applications of our visualization approach, this artifact description centers on the practical implementation.

The document is organized into the following key sections:

- an introduction of our main feature set,
- a brief overview of the technical implementation as well as the structure of or code base,
- a walkthrough of 5 concrete bytecode traces demonstrating the features of our visualization tool.

Where applicable, references to relevant sections of the paper are provided to connect the described features to their theoretical foundations.

II. Included Artifacts

Our artifact includes the main web-based visualization tool (located in `visualizer.zip`) as well as a set of demonstration bytecode traces (located in `demos.zip`). These bytecode traces have been generated using a modified version of the *MicroJava* VM (see [1], Section II-C). The demo traces should serve as a good testing ground for our project. More details on MicroJava (not needed to exercise this artifact) are given in [2] and a reference compiler and VM for MicroJava can be found at [3].

III. Tool Functionality

The functionality of our visualization tool is described in detail in our main publication. This section outlines the core functionality that will serve as the basis for this technical overview.

- *Interactive Execution Playback*: Users can step through the execution trace of a program's bytecode both forward and backward. This enables detailed inspection of the interpretation of a program at each operation, supporting fine-grained program analysis, and enhancing comprehension (see [1], Section IV).
- *Jump and Playback Controls*: Users can jump directly to specific bytecode or source code positions, or use a slider and control buttons to navigate through the execution. An optional "play until" feature animates execution until a selected target is reached.
- *Combined Code Visualization*: The tool displays the original high-level source code along with its compiled bytecode. It dynamically highlights the current position in the source code and bytecode. Our tool supports special visualizations for control flow, such as arrows indicating the destinations of jump operations (see [1], Section V).

This document is an extension of "*Trace-Based Bytecode Interpreter Visualization for Compiler Construction Education*" by Herber and Weninger, describing the artifacts of said work [1]. A video accompanying this artifact can also be found at https://ssw.jku.at/General/Staff/Weninger/Projects/InterpreterViz/VISSOFT25/Video.mp4.

- *Animated Memory Visualization*: The interpreter's memory regions (stack, heap, globals, etc.) are shown as distinct animated views. Memory changes (allocations, mutations, arithmetic, control flow) are visualized in real time, with support for displaying symbol information such as variable names and types.
- *Execution Trace Format*: The underlying execution trace format is log-structured and allows for easy extension of our visualization to other bytecode formats. It includes a recoding of all executed operations, including, the operation and its operands, updated memory pointers, and additional values, such as the computed results of an arithmetic operation, or the source and destination addresses of a move operation.
- *Customizable Visualization via DSL*: A domain-specific language (DSL) defines the semantics of bytecode operations and the structure of memory regions. This allows the tool to support other bytecode formats beyond MicroJava, and provides a clean abstraction layer for extending visualization behavior.
- *Reference Bytecodes*: The tool highlights mismatches between actual and expected bytecode sequences, supporting comparison against reference implementations. This feature can aid students in debugging their own faulty compiler implementations.
- *Interpreter Error Reporting*: Runtime errors are captured during trace collection and displayed in the visualization, enabling diagnosis of the execution leading up to an interpreter error.

These features make the tool a powerful resource for both educators and students in compiler construction and provide an accessible platform for exploring the behavior of bytecode interpreters through interactive, animated execution traces.

IV. Developer Overview

In this section, we provide an overview of our project's structure as well as select implementation components. Moreover, we describe the steps to start a development version of our visualization tool using the provided artifacts.

A. Core Implementation Details

Our visualization is written in Typescript[1], a statically typed superset of JavaScript, which allows us to perform compile-time type checking on JavaScript code. The user interface (UI) is implemented using the React[2] framework, which allows us to write component-based UIs by writing markup directly in TypeScript (so-called JSX [3]). While we implemented our own animation orchestration framework, the underlying animations are executed using the Framer Motion[4] library. The Vite-bundler [5] is used to transform developer-written TypeScript code into highly optimized JavaScript code which browsers can execute natively.

[1]Typescript https://www.typescriptlang.org/

[2]React https://react.dev/

[3]JSX in Typescript https://www.typescriptlang.org/docs/handbook/jsx.html

[4]Framer Motion https://www.npmjs.com/package/framer-motion

[5]Vite https://vite.dev/

B. File Structure

The compressed folder `visualizer.zip` contains our web-based visualization project. This folder contains a `package.json` file which describes the build and start scripts of our project as well as the dependencies needed to to run it. The `src` folder contains the TypeScript files used to implement our tool.

The source code is grouped into several modules located in the `src/modules` folder:

- The `lib` module which includes several utility functions, mostly for manipulating the browser's DOM tree [6] as well as functions for managing application state.
- The `memory` module implements our memory snapshot functionality. The state of our visualization is supported by memory snapshots. These memory snapshots are generated by the visualization specification DSL implementation. The developer of a visualization specification can simply manipulate a memory snapshot to define the behavior of each bytecode operation.
- The `operation-trace` module implements the parser for our trace format. This format is implemented as a log-structure which contains various JSON objects describing the actual data. Our parser returns a stream that allows us to start processing (and rendering the visualization) even before the entire trace has been processed. This can greatly increase the user-perceived performance especially for large traces.
- The `vizspec` module implements the parser for our visualization specification. Due to the simplicity of our format, the parser and the scanner are implemented in the same function. The entire implementation is about 200 lines of code.
- The `runner` module receives the stream from the `operation-trace` module and processes each trace entry using the corresponding visualization specification from the `vizspec` module. Similarly to the `operation-trace` module, it returns a stream, which again enables us to render the UI before the entire trace has been processed.
- The `run-manager` module implements a multi-threaded Web Worker [7] which performs the actual processing of the bytecode trace using the **operation-trace** and `runner` modules. This multi-threaded design ensures that our UI (which is in a separate thread) remains responsive even for large and expensive-to-process traces.
- The `state` module computes and holds the state for the current visualization. Once a user navigates to a specific bytecode position, the state module requests the current and the previous memory snapshots from the `run-manager` module. It uses these snapshots to plan the animations that transition from the previous

[6]DOM https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

[7]Web Worker https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

snapshot to the current snapshot. Moreover, it updates the code visualization to highlight the current position in the bytecode and source code. Additionally, it computes complementary state, such as the data needed to display the jump arrows in case the current operation is a jump operation.
- The `visualization` module receives the state from the `state` module and reactively updates the visualization to match current state. Moreover, it also receives the animations from the `state` module, which it then executes in sequence.
- The `start` module renders the UI initially presented to users when they open our visualization tool. This UI prompts the user to either select a new bytecode trace file to visualize or to choose an existing bytecode trace from the history to revisit.

C. Project Setup

This section describes how to set up the artifacts attached to this document. Additionally, a hosted instance of our project is provided at https://ssw.jku.at/General/Staff/Weninger/Projects/InterpreterViz/VISSOFT25/demo.

a) 1) Manual build and run: Extract `visualizer.zip` and open a terminal in the extracted folder. With Node.js[8] and Yarn Classic[9] installed, build and serve the production bundle with:

```
yarn
yarn build
yarn preview --host 0.0.0.0 --port 3000
```

The visualizer is then available at http://localhost:3000. For development, `yarn dev` starts Vite directly, normally at http://localhost:5173.

b) 2) Local Docker run: As a separate local option, install Docker[10] and run one container without Docker Compose:

```
docker build -t visualizer .
docker run --rm -p 3000:3000 visualizer
```

The container serves the visualizer at http://localhost:3000.

c) 3) Long-term run on a server (Docker Compose): Commits to https://github.com/SSW-JKU/univiz-interpreterviz publish `latest` and commit-specific images to ghcr.io/ssw-jku/univiz-interpreterviz. The repository's `deploy/docker-compose.yml` runs `interpreterviz-app` on host loopback port 3004 and `interpreterviz-watchtower`, which checks for a new labeled image every 120 seconds. On a Linux server with Docker Engine and its Compose plugin, run:

```
sudo mkdir -p /opt/interpreterviz
cd /opt/interpreterviz
sudo curl -fsSL -o docker-compose.yml \
  https://raw.githubusercontent.com/SSW-JKU/univiz-
     interpreterviz/HEAD/deploy/docker-compose.yml
sudo docker compose pull
sudo docker compose up -d --remove-orphans
sudo docker compose ps
```

[8]Node.js https://nodejs.org/en

[9]Yarn Classic https://classic.yarnpkg.com/en/docs

[10]Docker https://www.docker.com/

The loopback binding prevents direct public access. Any HTTPS reverse proxy can expose the application. For example, if the host uses Caddy[11], add the included `Caddyfile.example` block (replace the domain if needed):

```
interpreter.univiz.org {
  reverse_proxy 127.0.0.1:3004
}
```

After DNS points to the server, add the block to Caddy and reload it. If the host uses nginx instead, configure its HTTPS server to proxy to `http://127.0.0.1:3004`; the repository README provides complete examples for both reverse proxies.

V. A Guided Tour Through our Visualization Tool

To illustrate the features and capabilities of our trace-based bytecode interpreter visualization, we provide five curated demonstration files which walk through key aspects of our system. These examples are arranged to align with the progression of details described in our paper. Starting from basic concepts such as arithmetic operations and memory layout, then moving towards more complex features such as control flow, interpreter error handling, and bytecode discrepancies.

A. The Start Page

Upon opening our visualization tool (users may set up their own instance or use our hosted instance at https://ssw.jku.at/General/Staff/Weninger/Projects/InterpreterViz/VISSOFT25/demo), users are presented with the start page prompting them to select a bytecode trace. Once a bytecode trace has been selected, the tool will display the main visualization. After the first bytecode trace has been loaded, the start page also includes a history panel allowing users to quickly revisit bytecode traces that they have examined previously.

B. Loading Trace File

There are two ways to load an execution trace file: 1) clicking the dashed box on the start page, which will open the system's file selection dialog, prompting the user to select a trace file, or 2) dragging a file over the start page. After a file is selected, our visualization will immediately begin processing it. After the first operations are processed (which usually takes about `100ms`) the tool navigates to the main visualization.

To enable users to quickly and easily explore our visualization tool without having to obtain a trace file first, five demo execution traces (which are detailed in the following sections) are built into the hosted instance of our tool available at https://ssw.jku.at/General/Staff/Weninger/Projects/InterpreterViz/VISSOFT25/demo. These traces have also been used to demonstrate the tool's feature set in the original paper [1]. Directly providing these demo traces within our tool enables people to test our visualization tool and verify the claims in our paper.

[11]Caddy https://caddyserver.com/

C. Arithmetic Operations (see [1], Sections V & VI)

The *Arithmetic* demo presents a simple program which performs basic arithmetic operations using local and global variables. This serves as a good starting point to get familiar with the layout and interaction model of our tool.

Upon loading the demo, users are presented with the code visualization on the left and the memory visualization on the right Figure 1. As described in [1], Section V, the code view displays the high-level source code next to its corresponding bytecode. Stepping through the trace with the navigation controls (see [1], Section VII-A) highlights both the executed bytecode operation and its corresponding source code, allowing users to follow which bytecode operations stem from which source code.

Meanwhile, the memory visualization (see [1], Section VI-A) dynamically displays memory state transitions across interpreter regions. In this demo, users can observe how values are loaded, added/multiplied, and stored between local variables and statics, with animated transitions indicating movement and mutation of memory entries.

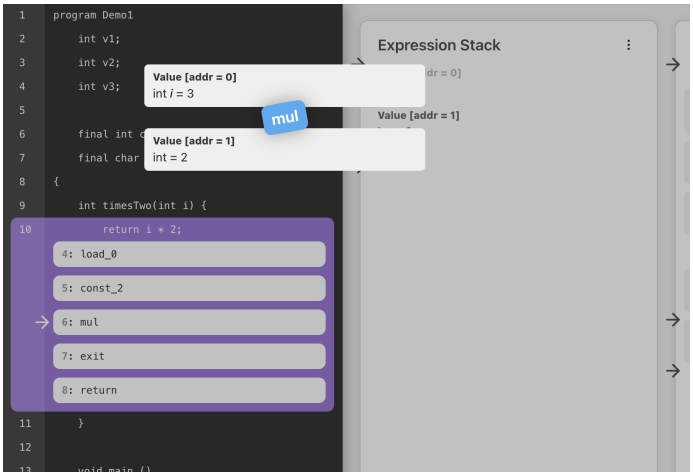

Fig. 1: An in-progress multiplication operation animation symbolizing how two input memory entries are merged together to produce a single output memory entry.

D. Heap Allocation (see [1], Section VI-B)

The *Heap Allocation* demo builds on the previous example by introducing dynamic memory management. The program performs object and array allocations, populating the heap with structured data.

As detailed in [1], Section VI-B, reference memory entries are visualized with dashed borders. When hovering over a reference entry, the visualization highlights the referenced memory region, making pointer relationships more intuitive.

Additionally, the memory entries that make up memory structures, such as arrays and objects, are grouped together with enclosing boxes and type labels (see [1], Section VI-D), as depicted in Figure 2. Instead of presenting the memory entries individually, this allows users to visually associate memory entries with their underlying structures.

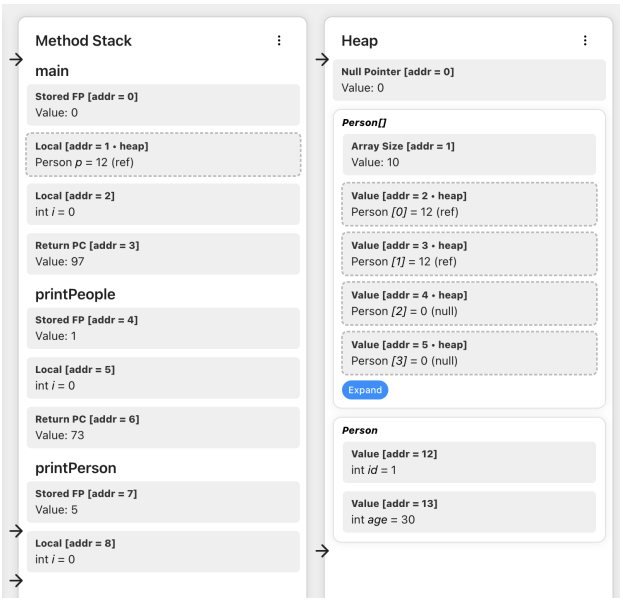

Fig. 2: The method stack and heap regions. The heap region contains a `Person` array as well as a `Person` object.

E. Control Flow (see [1], Section V-B)

The *Control Flow* demo contains a trace of a program which implements a complicated conditional statement using short-circuit evaluation. This highlights our tool's capability to visualize complex control flow behavior, as outlined in [1], Section V-B.

Jump operations in the bytecode are visualized using arrows that connect source and destination instructions, as shown in Figure 3. All branches of the control structure (e.g., if, else, elseif) are marked collectively, making their relationship easily understandable.

This example is particularly useful for understanding how high-level control constructs are compiled into low-level jumps and how they are optimized using short-circuit evaluation.

F. Interpreter Error (see [1], Section IV-C)

The *Interpreter Error* demo show how our visualization handles faulty bytecode and bytecode interpreter failures. As described in [1], Section IV-C, the trace includes error metadata that is presented in the visualization at the point of failure.

When stepping through this example, users will encounter an erroneous operation. Specifically, a stack underflow, where our compiler has generated two `ADD` operations instead of one, which means that the expression stack does not have the required two input memory entries for the second `ADD` operation. The visualization not only halts execution at this point, but also highlights the faulty instruction and its memory context, helping users find the root cause. Moreover, each error code has an associated hint which briefly describes what the error could mean, as depicted in Figure 4.

This feature is especially useful for students debugging their own compiler implementations, offering concrete visual

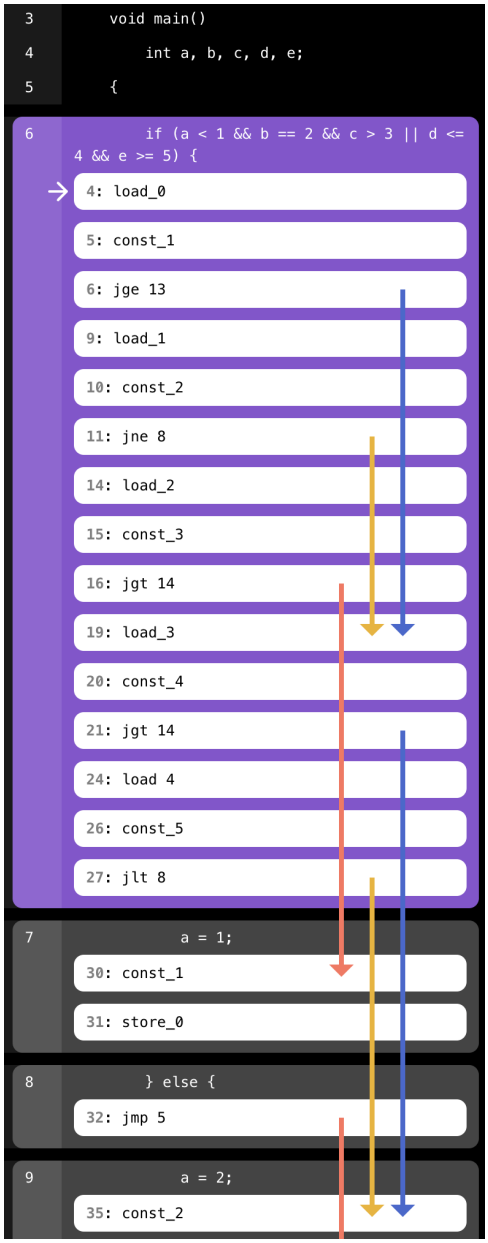

Fig. 3: A complex if statement and the complex control flow operation used to implement it.

feedback rather than abstract error messages. It should also be noted that bytecode errors are often more complex than a single invalid operation. Our visualization solves this by allowing users to step through the entire execution leading up to the error, thereby enabling concrete exploration of erroneous bytecode.

G. *Reference Bytecode (see [1], Section V-C)*

Finally, the *Reference Bytecode* demo illustrates our support for highlighting the discrepancy between expected and actual bytecode sequences. As detailed in [1], Section V-C, this feature compares the student-generated bytecode against a reference implementation and highlights divergences directly within the user interface.

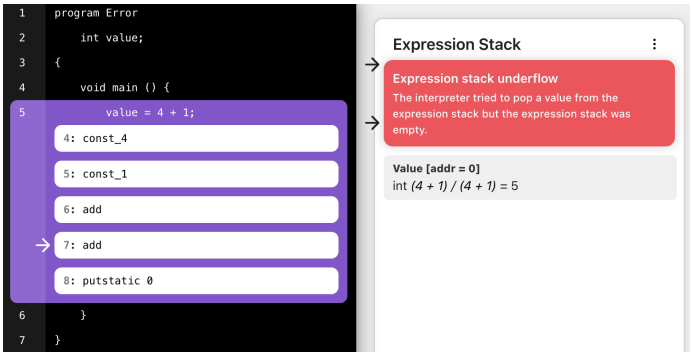

Fig. 4: An erroneous bytecode operation and the corresponding interpreter error displayed in the expression stack memory region.

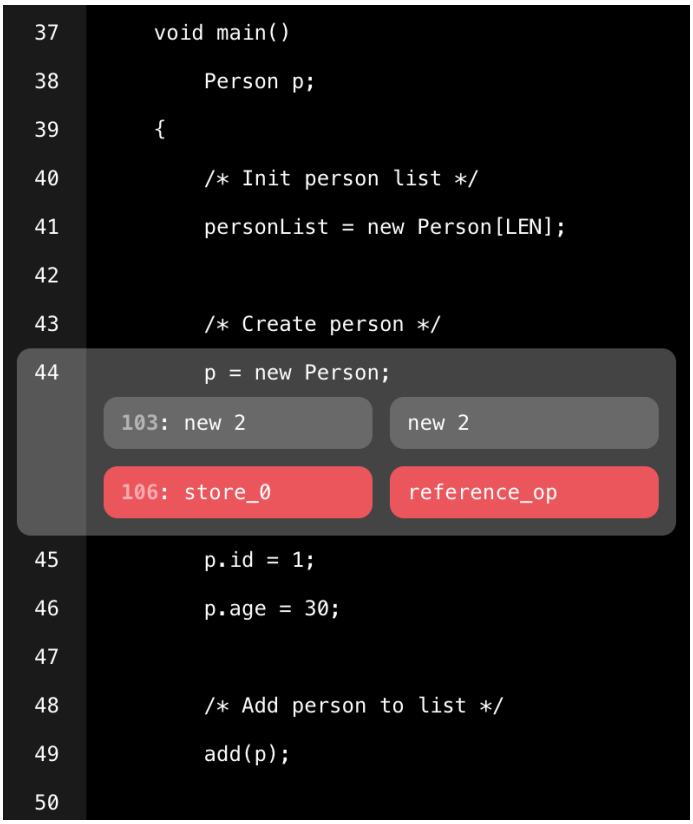

Fig. 5: The code visualization where a single bytecode operation is highlighted because it does not match the reference bytecode.

In this example, users see the actual and reference bytecode listed in parallel columns. Invalid instructions are colored red, and their origin is linked back to the source code using the bytecode-to-line mapping, as displayed in Figure 5. This comparison enables students to immediately identify whether incorrect bytecode was generated.

These bytecode traces provide concrete examples of our visualization tool's core functionality. We encourage readers to try our hosted instance at https://ssw.jku.at/General/Staff/Weninger/Projects/InterpreterViz/VISSOFT25/demo.

References

[1] T. Herber and M. Weninger, "Trace-Based Bytecode Interpreter Visualization for Compiler Construction Education," in *Working Conf. on Software Visualization (VISSOFT) – in print*.

[2] H. Mössenböck, *Compiler Construction: Fundamentals and Applications*. Springer Nature Switzerland, 2025. [Online]. Available: https://link.springer.com/book/10.1007/978-3-031-84813-1

[3] ——. (2025) Compiler Construction: Fundamentals and Applications (MicroJava Compiler and MicroJava VM Reference Implementation). [Online]. Available: https://ssw.jku.at/CompilerBook/